📋 MEDFLOW — DOCUMENTAÇÃO TÉCNICA COMPLETA E DETALHADA

Sistema de Gestão de Clínica Médica

> **Documento Privado — Contém credenciais, rotas internas e toda a lógica do sistema.**
> Versão: 2.9 | Data: 29 de Setembro de 2026
> Criado para pessoas que **nunca tiveram contato com programação**. Cada conceito é explicado do zero.

PARTE 1 — VISÃO GERAL: O QUE É O MEDFLOW?

1.1. O que este sistema faz?

O MedFlow é um **sistema completo de gestão de clínica médica** construído do zero. Ele funciona via internet (como um site) e gerencia todo o ciclo de vida de uma consulta médica: desde o momento em que o paciente se cadastra, passa pela recepção, faz triagem, é atendido pelo médico, recebe receitas e atestados, até o pagamento e faturamento final. Além disso, o sistema registra um rastro de auditoria de tudo que acontece.

Imagine uma clínica real. Nela existem diferentes pessoas com diferentes funções:

- O **Paciente** que vem para ser atendido
- A **Recepcionista** que agenda e faz check-in
- O **Enfermeiro de Triagem** que mede pressão, temperatura, etc.
- O **Médico** que atende, prescreve e faz diagnóstico
- O **Setor de Faturamento** que cuida dos pagamentos
- O **Administrador** que gerencia tudo

O MedFlow é o software que conecta todas essas pessoas em um único lugar, com controle de acesso (cada pessoa só vê o que pode ver) e rastreabilidade total.

1.2. Como o sistema está organizado?

O MedFlow usa uma arquitetura chamada **"Cliente-Servidor"** com microserviços containerizados. Vou explicar cada termo:

O que é "Cliente-Servidor"?

Pense em um restaurante. O **cliente** (frontend) é o cardápio bonito que fica na mesa — é o que o usuário vê e interage. O **servidor** (backend) é a cozinha — é onde toda a lógica e o trabalho pesado acontecem. O garçom (a internet/rede) leva os pedidos do cardápio para a cozinha e traz a comida de volta.

O que são "Containers" (Docker)?

Imagine que você quer montar uma loja. Em vez de construir do zero, você usa **containers de transporte** (aqueles de navio) que já vêm prontos com tudo dentro. Docker faz exatamente isso com software: ele empacota cada parte do sistema em uma "caixa" isolada que funciona igualzinho em qualquer computador. Isso elimina o problema de "na minha máquina funciona".

As "caixas" (containers) do MedFlow:

1. **Backend** (a cozinha) — Aplicação Java/Spring Boot
2. **Frontend** (o cardápio) — Aplicação React/TypeScript
3. **Banco de Dados** (o estoque) — PostgreSQL
4. **Mensageria** (o interfone entre setores) — RabbitMQ

5. **Monitoramento** (câmeras de segurança) — Prometheus + Grafana

---

PARTE 2 — TECNOLOGIAS: O QUE É CADA PROGRAMA E PRA QUE SERVE

2.1. Linguagens de Programação

Java 21

- **O que é?** Uma das linguagens de programação mais usadas no mundo, especialmente para sistemas empresariais e bancários. Foi criada pela Sun Microsystems em 1995 e hoje pertence à Oracle.
- **Pra que serve aqui?** É a linguagem do backend (servidor). Todo o código que processa dados, valida regras de negócio, se comunica com o banco de dados e gera as respostas é escrito em Java.
- **Por que Java?** É robusta (poucos erros), segura (muito usada em bancos), e tem uma comunidade gigante de desenvolvedores.
- **Versão 21:** É a versão mais recente com suporte de longo prazo (LTS), o que significa que vai receber atualizações de segurança por anos.

TypeScript

- **O que é?** Uma versão "melhorada" do JavaScript (a linguagem que todo site usa). O TypeScript adiciona "tipos" — ou seja, você define que uma variável é um número, um texto, etc., e o sistema avisa se você usar errado.
- **Pra que serve aqui?** É a linguagem do frontend (interface do usuário). Tudo que o usuário vê — botões, formulários, tabelas — é feito com TypeScript.

SQL (Structured Query Language)

- **O que é?** A linguagem para conversar com bancos de dados. É como falar com o estoquista: "Me dá todos os pacientes que têm consulta hoje" ou "Cadastra esse novo médico".
- **Pra que serve aqui?** Criar as tabelas do banco, fazer consultas, inserir dados.

HTML e CSS

- **O que é?** HTML é a linguagem que define a estrutura de uma página web (títulos, parágrafos, botões). CSS é o que define a aparência (cores, tamanhos, posições).
- **Pra que serve aqui?** São a base visual do frontend.

2.2. Frameworks (Ferramentas Prontas)

Spring Boot 4.1.1

- **O que é?** Um framework (kit de ferramentas pronto) para Java. Em vez de escrever tudo do zero, o Spring Boot já traz pronto: servidor web, conexão com banco de dados, segurança, etc. É como comprar uma casa pré-fabricada em vez de construir tijolo por tijolo.

- **Pra que serve aqui?** É o coração do backend. Ele:
  - Recebe as requisições HTTP (pedidos do frontend)
  - Processa as regras de negócio
  - Se comunica com o banco de dados
  - Retorna as respostas em JSON

Spring Security

- **O que é?** O módulo de segurança do Spring. Ele cuida de tudo relacionado a autenticação ("quem é você?") e autorização ("o que você pode fazer?").
- **Pra que serve aqui?** Garante que:
  - Ninguém acessa o sistema sem fazer login
  - Um paciente não vê dados de outro paciente
  - Só médicos podem criar prontuários
  - Só admins podem ver a auditoria

Spring Data JPA (Java Persistence API)

- **O que é?** Um módulo que facilita enormemente a comunicação com o banco de dados. Em vez de escrever SQL na mão, você simplesmente define objetos Java e o Spring gera o SQL automaticamente.
- **Pra que serve aqui?** Converter entre objetos Java (Paciente, Médico, Consulta) e as tabelas do banco de dados PostgreSQL.

React 18

- **O que é?** Uma biblioteca JavaScript criada pelo Facebook para construir interfaces de usuário. É usada por Instagram, WhatsApp Web, Netflix, Airbnb e milhares de outros sites.
- **Pra que serve aqui?** Construir todas as telas que o usuário vê: login, dashboard, lista de médicos, consultas, etc.

Tailwind CSS

- **O que é?** Um framework de CSS que oferece classes prontas para estilizar rapidamente. Em vez de escrever `color: blue; font-size: 16px;`, você escreve `text-blue-500 text-base` direto no HTML.
- **Pra que serve aqui?** Deixar o frontend bonito de forma rápida e consistente.

Vite

- **O que é?** Uma ferramenta moderna para desenvolvimento frontend. Ela faz o "build" (compilação) do código TypeScript/React para algo que o navegador entende, e fornece um servidor de desenvolvimento com atualização instantânea.
- **Pra que serve aqui?** Compilar e servir o frontend durante o desenvolvimento.

2.3. Banco de Dados

PostgreSQL 16

- **O que é?** Um dos bancos de dados relacionais mais poderosos e confiáveis do mundo. É gratuito e open-source. Grandes empresas como Apple, Spotify e Instagram usam PostgreSQL.
- **Pra que serve aqui?** Guardar TODOS os dados do sistema de forma permanente:
  - Cadastro de usuários, pacientes, médicos
  - Consultas agendadas
  - Prontuários médicos
  - Receitas e atestados
  - Pagamentos
  - Logs de auditoria
- **Por que PostgreSQL?** É extremamente confiável, suporta milhões de registros, tem recursos avançados (JSON, full-text search, UUIDs nativos) e é gratuito.
- **Credenciais configuradas:**
  - Usuário: `medflow`
  - Senha: `medflow`
  - Banco: `medflow`
  - Porta: `5432`

Flyway

- **O que é?** Uma ferramenta de "migração" de banco de dados. Ela garante que a estrutura do banco (tabelas, colunas) evolua de forma controlada e rastreável, como um controle de versão para o banco.
- **Pra que serve aqui?** Cada vez que o sistema sobe, o Flyway verifica se existem novas migrações (arquivos SQL numerados) que ainda não foram executadas. Se existem, ele executa na ordem. Isso garante que o banco está sempre atualizado, sem precisar recriar nada manualmente.
- **Migrações do MedFlow (na ordem de execução):**
  - `V1` — Criar tabelas iniciais (usuario, paciente, medico, consulta, etc.)
  - `V2` — Criar tabela de funcionários (recepcionista, faturamento)
  - `V3` — Criar tabela de especialidades médicas (M:N)
  - `V4` — Remover coluna antiga de especialidade do médico
  - `V5` — Criar tabela de triagem
  - `V6` — Criar tabelas de receitas, atestados, e evoluir prontuário
  - `V7` — Criar tabela de auditoria

2.4. Mensageria

RabbitMQ

- **O que é?** Um sistema de filas de mensagens. Imagine uma caixa de correio: em vez de entregar uma carta na mão da pessoa (que pode não estar em casa), você coloca na caixa e ela pega quando puder. RabbitMQ faz isso com mensagens entre diferentes partes do sistema.

- **Pra que serve aqui?** Quando uma consulta é agendada, remarcada, cancelada ou confirmada, o sistema coloca uma mensagem na fila `notificacoes`. Um "ouvinte" (listener) no backend consome essas mensagens e pode enviar e-mails, SMS, push notifications, etc. Hoje ele só imprime no console, mas a infraestrutura está pronta para qualquer canal de notificação.
- **Credenciais:**
  - Usuário: `guest`
  - Senha: `guest`
  - Porta AMQP: `5672`
  - Porta do Painel Web: `15672` (acesse http://localhost:15672 para ver as filas)

2.5. Monitoramento

Prometheus

- **O que é?** Um sistema de monitoramento de código aberto criado pelo SoundCloud. Ele coleta métricas numéricas do sistema ao longo do tempo: quantas requisições por segundo, quanto de memória está usando, tempo de resposta, etc.
- **Pra que serve aqui?** Ele faz "scraping" (coleta) das métricas que o Spring Boot Actuator expõe no endpoint `/actuator/prometheus`. Isso inclui:
  - Número de requisições HTTP por endpoint
  - Tempo de resposta de cada requisição
  - Uso de memória da JVM (máquina virtual Java)
  - Status do banco de dados
  - Filas do RabbitMQ
- **Porta:** `9090` (acesse http://localhost:9090 para ver as métricas)

Grafana

- **O que é?** Uma plataforma de visualização de dados. Enquanto o Prometheus coleta os números, o Grafana transforma esses números em gráficos bonitos e dashboards interativos.
- **Pra que serve aqui?** Criar dashboards visuais para acompanhar a saúde do sistema em tempo real:
  - Gráficos de uso de CPU e memória
  - Taxa de erros vs. taxa de sucesso
  - Tempo médio de resposta das APIs
  - Alertas quando algo está errado
- **Credenciais:**
  - Usuário: `admin`
  - Senha: `admin`
  - Porta: `3000` (acesse http://localhost:3000)

2.6. Containerização e Orquestração

Docker

- **O que é?** Uma plataforma que permite empacotar aplicações em "containers" — ambientes isolados e leves que contêm tudo que a aplicação precisa para rodar (código, bibliotecas, configurações).
- **Pra que serve aqui?** Cada componente do MedFlow (backend, frontend, banco, rabbitmq, prometheus, grafana) roda em seu próprio container Docker. Isso garante:
  - Isolamento: um componente não interfere no outro
  - Portabilidade: funciona igualzinho em qualquer máquina
  - Reprodutibilidade: qualquer pessoa pode rodar o sistema com um único comando

Docker Compose

- **O que é?** Uma ferramenta que permite definir e rodar múltiplos containers Docker de uma vez só, usando um arquivo de configuração (`docker-compose.yml`).
- **Pra que serve aqui?** Com um único comando (`docker compose up`), todos os 6 containers sobem juntos, na ordem correta, com todas as configurações. Sem ele, teríamos que subir cada container manualmente.

Kubernetes (K8s)

- **O que é?** Um sistema de orquestração de containers criado pelo Google. Se o Docker Compose é como ter 6 jarros de plantas na janela, o Kubernetes é como ter uma estufa automatizada com irrigação, controle de temperatura e replantio automático.
- **Pra que serve aqui?** O MedFlow tem arquivos de configuração Kubernetes (na pasta `k8s/`) preparados para deploy em produção. O Kubernetes oferece:
  - **Escalabilidade automática:** Se muitos pacientes acessam ao mesmo tempo, ele cria mais cópias do backend automaticamente
  - **Auto-recuperação:** Se um container morre, o Kubernetes reinicia sozinho
  - **Load balancing:** Distribui o tráfego entre as cópias
  - **Rolling updates:** Atualiza o sistema sem downtime (sem ficar fora do ar)
- **Arquivos K8s do MedFlow:**
  - `k8s/backend.yaml` — Deploy do backend (2 réplicas)
  - `k8s/frontend.yaml` — Deploy do frontend (2 réplicas)
  - `k8s/postgres.yaml` — Deploy do banco de dados (com PersistentVolumeClaim para não perder dados)
  - `k8s/rabbitmq.yaml` — Deploy do RabbitMQ

2.7. Outras Ferramentas

Maven

- **O que é?** Um gerenciador de dependências e build para Java. Pense nele como um gerente de compras: você lista tudo que precisa (bibliotecas) em um arquivo (`pom.xml`) e ele baixa, organiza e compila tudo para você.

- **Pra que serve aqui?** Gerenciar todas as bibliotecas do backend (Spring, JWT, PostgreSQL driver, etc.) e compilar o código Java em um arquivo executável (.jar).

Lombok

- **O que é?** Uma biblioteca Java que elimina código repetitivo ("boilerplate"). Em vez de escrever 50 linhas de getters, setters e construtores, você coloca uma anotação `@Data` e o Lombok gera tudo automaticamente.
- **Pra que serve aqui?** Todas as entidades e DTOs usam `@Data` (gera getters/setters), `@RequiredArgsConstructor` (gera construtor com dependências).

JWT (JSON Web Token)

- **O que é?** Um padrão aberto para criar tokens de autenticação seguros. Funciona como um crachá digital: quando você faz login, o sistema gera um token (uma string longa e criptografada) que contém seus dados (email, role). Esse token é enviado em todas as próximas requisições para provar sua identidade.
- **Pra que serve aqui?** Autenticação do sistema. Após o login, o frontend guarda o token JWT no localStorage e o envia em cada requisição no header `Authorization: Bearer <token>`.

Swagger / OpenAPI (SpringDoc)

- **O que é?** Uma ferramenta que gera documentação interativa da API automaticamente. Ela lê o código e cria uma página web onde você pode ver todas as rotas, parâmetros e até testar as requisições direto no navegador.
- **Pra que serve aqui?** Acesse http://localhost:8080/swagger-ui.html para ver e testar todas as APIs do MedFlow sem precisar de nenhuma ferramenta adicional.

Micrometer

- **O que é?** Uma biblioteca de instrumentação para Java que coleta métricas da aplicação e as expõe em formatos que o Prometheus entende.
- **Pra que serve aqui?** Expõe métricas do Spring Boot (requisições, erros, tempo de resposta, memória) no endpoint `/actuator/prometheus`.

Nginx

- **O que é?** Um servidor web de altíssima performance. É usado por metade dos sites mais visitados do mundo.
- **Pra que serve aqui?** No container do frontend, o Nginx serve os arquivos estáticos (HTML, CSS, JS) do React e faz proxy reverso para o backend, redirecionando chamadas de `/api/*` para o servidor Java.

Git

- **O que é?** Um sistema de controle de versão. Ele rastreia todas as mudanças feitas no código ao longo do tempo, permitindo voltar para versões anteriores, trabalhar em equipe, e manter um histórico completo.
- **Pra que serve aqui?** Todo o código do MedFlow está versionado no GitHub: https://github.com/Samuka310/medflow

---

PARTE 3 — ESTRUTURA DE PASTAS: O QUE É CADA PASTA E POR QUE ESTÁ ASSIM

```
Medflow/                          ← RAIZ DO PROJETO
│
├── docker-compose.yml             ← Arquivo que orquestra TODOS os containers
│
├── backend/                       ← SERVIDOR (a "cozinha" do sistema)
│   ├── Dockerfile                 ← Receita para construir o container do backend
│   ├── pom.xml                    ← Lista de dependências Java (Maven)
│   └── src/
│       ├── main/
│       │   ├── java/com/medflow/
│       │   │   ├── MedflowApplication.java     ← Ponto de partida do sistema
│       │   │   ├── config/                      ← Configurações do sistema
│       │   │   │   ├── AppConfig.java           ← Config geral (PasswordEncoder)
│       │   │   │   ├── CorsConfig.java          ← Config de Cross-Origin (CORS)
│       │   │   │   ├── OpenApiConfig.java       ← Config do Swagger
│       │   │   │   ├── RabbitMQConfig.java      ← Config das filas RabbitMQ
│       │   │   │   └── SecurityConfig.java      ← Config do Spring Security
│       │   │   ├── security/                    ← Segurança (JWT)
│       │   │   │   ├── JwtService.java          ← Gera e valida tokens JWT
│       │   │   │   └── JwtAuthenticationFilter.java  ← Filtro que intercepta requisições
│       │   │   ├── entity/                      ← Entidades (representam tabelas do banco)
│       │   │   │   ├── Usuario.java
│       │   │   │   ├── Paciente.java
│       │   │   │   ├── Medico.java
│       │   │   │   ├── Funcionario.java
│       │   │   │   ├── Consulta.java
│       │   │   │   ├── Prontuario.java
│       │   │   │   ├── Pagamento.java
│       │   │   │   ├── Especialidade.java
│       │   │   │   ├── Exame.java
│       │   │   │   ├── Triagem.java
│       │   │   │   ├── Receita.java
│       │   │   │   ├── Atestado.java
│       │   │   │   └── Auditoria.java
│       │   │   ├── dto/                         ← Objetos de transferência de dados
│       │   │   │   ├── LoginRequest.java
│       │   │   │   ├── LoginResponse.java
│       │   │   │   ├── RegisterRequest.java
│       │   │   │   ├── ConsultaDTO.java
│       │   │   │   ├── PacienteDTO.java
│       │   │   │   ├── PacienteClinicoDTO.java
│       │   │   │   ├── MedicoDTO.java
│       │   │   │   ├── ProntuarioDTO.java
│       │   │   │   ├── PagamentoDTO.java
│       │   │   │   ├── ExameDTO.java
│       │   │   │   ├── EspecialidadeDTO.java
│       │   │   │   ├── TriagemDTO.java
│       │   │   │   ├── ReceitaDTO.java
│       │   │   │   ├── AtestadoDTO.java
│       │   │   │   ├── AuditoriaDTO.java
│       │   │   │   ├── FaturamentoRelatorioDTO.java
│       │   │   │   └── FuncionarioRequest.java
│       │   │   ├── repository/                  ← Interfaces de acesso ao banco
│       │   │   │   ├── UsuarioRepository.java
│       │   │   │   ├── PacienteRepository.java
│       │   │   │   ├── MedicoRepository.java
│       │   │   │   ├── ConsultaRepository.java
│       │   │   │   ├── ProntuarioRepository.java
│       │   │   │   ├── PagamentoRepository.java
│       │   │   │   ├── EspecialidadeRepository.java
│       │   │   │   ├── ExameRepository.java
│       │   │   │   ├── FuncionarioRepository.java
│       │   │   │   ├── TriagemRepository.java
│       │   │   │   ├── ReceitaRepository.java
│       │   │   │   ├── AtestadoRepository.java
```

```
│   │   │   │   │   └── AuditoriaRepository.java
│   │   │   │   ├── service/                    ← Lógica de negócio
│   │   │   │   │   ├── AuthService.java
│   │   │   │   │   ├── AdminService.java
│   │   │   │   │   ├── ConsultaService.java
│   │   │   │   │   ├── PacienteService.java
│   │   │   │   │   ├── MedicoService.java
│   │   │   │   │   ├── ProntuarioService.java
│   │   │   │   │   ├── PagamentoService.java
│   │   │   │   │   ├── EspecialidadeService.java
│   │   │   │   │   ├── ExameService.java
│   │   │   │   │   ├── TriagemService.java
│   │   │   │   │   ├── FaturamentoService.java
│   │   │   │   │   ├── AuditoriaService.java
│   │   │   │   │   └── NotificacaoListener.java
│   │   │   │   ├── controller/                 ← Portas de entrada (rotas HTTP)
│   │   │   │   │   ├── AuthController.java
│   │   │   │   │   ├── AdminController.java
│   │   │   │   │   ├── ConsultaController.java
│   │   │   │   │   ├── PacienteController.java
│   │   │   │   │   ├── MedicoController.java
│   │   │   │   │   ├── ProntuarioController.java
│   │   │   │   │   ├── PagamentoController.java
│   │   │   │   │   ├── EspecialidadeController.java
│   │   │   │   │   ├── ExameController.java
│   │   │   │   │   ├── FaturamentoController.java
│   │   │   │   │   └── AuditoriaController.java
│   │   │   │   └── exception/                  ← Tratamento de erros
│   │   │   │       ├── ErrorResponse.java
│   │   │   │       └── GlobalExceptionHandler.java
│   │   │   └── resources/
│   │   │       ├── application.yml              ← Config principal
│   │   │       ├── application-dev.yml           ← Config para desenvolvimento
│   │   │       ├── application-prod.yml          ← Config para produção
│   │   │       └── db/migration/                 ← Scripts de migração (Flyway)
│   │   │           ├── V1__criar_tabelas_iniciais.sql
│   │   │           ├── V2__criar_tabela_funcionario.sql
│   │   │           ├── V3__criar_tabela_medico_especialidade.sql
│   │   │           ├── V4__remover_coluna_especialidade_de_medico.sql
│   │   │           ├── V5__criar_tabela_triagem.sql
│   │   │           ├── V6__criar_receitas_atestados_evoluir_prontuario.sql
│   │   │           └── V7__criar_tabela_auditoria.sql
│   │   └── test/                                 ← Testes automatizados
│   │       └── java/com/medflow/
│   │           ├── MedflowApplicationTests.java
│   │           ├── SecurityIntegrationTest.java
│   │           └── service/
│   │               ├── AuthServiceTest.java
│   │               └── ConsultaServiceTest.java
│
├── frontend/                       ← INTERFACE DO USUÁRIO (o "cardápio")
│   ├── Dockerfile                  ← Receita para construir o container do frontend
│   ├── nginx.conf                  ← Configuração do servidor web Nginx
│   ├── package.json                ← Lista de dependências JavaScript
│   ├── vite.config.ts              ← Config do Vite (build tool)
│   ├── tsconfig.json               ← Config do TypeScript
│   ├── tailwind.config.js          ← Config do Tailwind CSS
│   ├── postcss.config.js           ← Config do PostCSS
│   ├── index.html                  ← Página HTML raiz
│   └── src/
│       ├── main.tsx                ← Ponto de entrada do React
│       ├── App.tsx                 ← Componente principal (rotas)
│       ├── App.css                 ← Estilos globais
│       ├── index.css               ← Estilos do Tailwind
│       ├── api/
│       │   └── axios.ts            ← Config do cliente HTTP (Axios)
│       ├── context/
│       │   └── AuthContext.tsx     ← Contexto de autenticação (estado global)
│       ├── components/
│       │   ├── Layout.tsx          ← Layout com menu lateral
│       │   └── ProtectedRoute.tsx ← Componente que protege rotas
│       └── pages/
│           ├── Login.tsx           ← Página de login
│           ├── Register.tsx        ← Página de cadastro
│           ├── Dashboard.tsx       ← Painel principal
```

```
│           ├── Medicos.tsx        ← Lista de médicos
│           └── MinhasConsultas.tsx ← Consultas do paciente
│
├── k8s/                          ← KUBERNETES (produção em nuvem)
│   ├── backend.yaml
│   ├── frontend.yaml
│   ├── postgres.yaml
│   └── rabbitmq.yaml
│
└── prometheus/                   ← MONITORAMENTO
    └── prometheus.yml            ← Config do Prometheus
```

Por que essa estrutura?

A organização segue o padrão **MVC (Model-View-Controller)** adaptado para Spring Boot:

- **Model (entity/):** Define como os dados são estruturados. Cada arquivo aqui representa uma tabela no banco de dados.
- **View (frontend/):** O que o usuário vê. Telas, botões, formulários.
- **Controller (controller/):** Recebe as requisições HTTP e delega para os serviços.
- **Service (service/):** Contém TODA a lógica de negócio. É onde as regras vivem.
- **Repository (repository/):** Interface de comunicação com o banco de dados.
- **DTO (dto/):** Objetos que definem exatamente quais dados entram e saem da API.
- **Config (config/):** Configurações do sistema (segurança, CORS, RabbitMQ).
- **Security (security/):** Tudo relacionado a autenticação JWT.
- **Exception (exception/):** Tratamento centralizado de erros.

PARTE 4 — ARQUITETURA DO BACKEND: CAMADA POR CAMADA

4.1. Camada de Entidades (entity/)

As entidades são classes Java que representam **exatamente** as tabelas do banco de dados. Cada campo da classe corresponde a uma coluna da tabela.

Usuario.java

É a entidade base de autenticação. Todo mundo que acessa o sistema (paciente, médico, recepcionista, admin) primeiro é um Usuário.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único universal (gerado automaticamente)
`nome`	String	Nome completo do usuário
`email`	String	E-mail (usado como login, deve ser único)
`senha`	String	Senha criptografada com BCrypt
`role`	String	Papel: ADMIN, PACIENTE, MEDICO, RECEPCIONISTA, TRIAGEM, FATURAMENTO

Implementa UserDetails do Spring Security, o que significa que o Spring sabe como usar esta classe para autenticação. O método `getAuthorities()` retorna a role com prefixo "ROLE_" (ex: `ROLE_ADMIN`), que é o padrão do Spring Security.

Paciente.java

Estende o conceito de Usuário para armazenar dados específicos de paciente.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único do paciente
`cpf`	String	CPF do paciente (único)
`dataNascimento`	LocalDate	Data de nascimento
`telefone`	String	Telefone de contato
`usuario`	Usuario	Referência ao usuário (ManyToOne)

Medico.java

Dados específicos de médico, incluindo CRM e valor da consulta.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`crm`	String	Registro no Conselho Regional de Medicina
`valorConsulta`	BigDecimal	Valor cobrado por consulta
`usuario`	Usuario	Referência ao usuário
`especialidades`	Set	Lista de especialidades (ManyToMany)

Funcionario.java

Para recepcionistas, técnicos de triagem e funcionários do faturamento.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`cargo`	String	RECEPCIONISTA, TRIAGEM ou FATURAMENTO
`dataAdmissao`	LocalDate	Data de admissão
`usuario`	Usuario	Referência ao usuário

Consulta.java

O coração do sistema. Cada consulta liga um paciente a um médico em uma data/hora.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`paciente`	Paciente	Quem vai ser atendido
`medico`	Medico	Quem vai atender
`dataHora`	LocalDateTime	Data e hora da consulta
`status`	String	AGENDADA, CONFIRMADA, AGUARDANDO_TRIAGEM, CANCELADA, REALIZADA

Fluxo de Status:

```
AGENDADA → CONFIRMADA → AGUARDANDO_TRIAGEM → REALIZADA
    ↓           ↓
CANCELADA    CANCELADA
```

Pagamento.java

Cada consulta gera automaticamente um pagamento pendente.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`consulta`	Consulta	A consulta associada (OneToOne)

Campo	Tipo	Descrição
`valor`	BigDecimal	Valor em reais
`status`	String	PENDENTE, PAGO, ESTORNADO
`formaPagamento`	String	CARTAO, PIX, DINHEIRO, etc.

Prontuario.java

Registro médico do atendimento.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`consulta`	Consulta	Consulta associada
`diagnosticoCid`	String	Código CID do diagnóstico
`evolucaoClinica`	String	Evolução clínica do paciente
`condutaMedica`	String	Conduta adotada pelo médico
`observacoes`	String	Observações gerais

Triagem.java

Dados de triagem (sinais vitais) coletados antes do atendimento médico.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`consulta`	Consulta	Consulta associada
`pressaoArterial`	String	Ex: "120/80"
`temperatura`	Double	Em graus Celsius
`frequenciaCardiaca`	Integer	Batimentos por minuto
`peso`	Double	Em kg
`altura`	Double	Em metros
`saturacao`	Integer	Saturação de oxigênio (%)
`queixaPrincipal`	String	O que o paciente está sentindo
`prioridade`	String	BAIXA, MEDIA, ALTA, EMERGENCIA

Especialidade.java

Especialidades médicas (Cardiologia, Dermatologia, etc.)

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único

Campo	Tipo	Descrição
`nome`	String	Nome da especialidade (único)

Exame.java

Exames associados a uma consulta.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`consulta`	Consulta	Consulta associada
`tipo`	String	Tipo do exame
`descricao`	String	Detalhes do exame
`resultado`	String	Resultado do exame
`status`	String	Status do exame

Receita.java

Prescrições médicas de medicamentos.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`paciente`	Paciente	Paciente que recebe
`medicamento`	String	Nome do medicamento
`dosagem`	String	Ex: "500mg"
`instrucoes`	String	Como tomar
`dataEmissao`	LocalDate	Data da emissão

Atestado.java

Atestados médicos emitidos pelo médico.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`paciente`	Paciente	Paciente
`descricao`	String	Motivo do atestado
`dias`	Integer	Quantidade de dias
`dataEmissao`	LocalDate	Data da emissão

Auditoria.java

Registro de todas as ações realizadas no sistema.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`usuario`	Usuario	Quem fez a ação
`usuarioEmail`	String	E-mail de quem fez
`acao`	String	AGENDAR_CONSULTA, EVENTO_PAGAMENTO, etc.
`entidade`	String	Consulta, Pagamento, Prontuario
`entidadeId`	UUID	ID do objeto afetado
`detalhes`	String	Descrição da ação
`dataHora`	LocalDateTime	Quando aconteceu

4.2. Camada de DTOs (dto/)

O que são DTOs? DTO significa "Data Transfer Object" (Objeto de Transferência de Dados). São objetos simples que definem exatamente quais dados a API recebe e retorna. Eles existem por segurança: em vez de expor a entidade inteira (que pode conter dados sensíveis como senha), o DTO filtra apenas os campos necessários.

Analogia: Se a entidade é o seu documento completo (RG, CPF, endereço, tudo), o DTO é o crachá que mostra só o que é necessário para aquele contexto.

LoginRequest.java

```
Campos: email, senha
Uso: Recebe credenciais do usuário no endpoint /auth/login
```

LoginResponse.java

```
Campos: token, role
Uso: Retorna o JWT e a role do usuário após login bem-sucedido
```

RegisterRequest.java

```
Campos: nome, email, senha, cpf, dataNascimento, telefone
Uso: Dados para cadastro de novo paciente em /auth/register
```

ConsultaDTO.java

```
Campos: id, medicoId, pacienteId, dataHora, status, pagamentoId
Uso: Agendamento e listagem de consultas
Validações: dataHora @Future (deve ser no futuro), medicoId e pacienteId @NotNull
```

PacienteDTO.java

```
Campos: id, cpf, dataNascimento, telefone, nomeUsuario, nome, email, senha
Uso: Cadastro e listagem de pacientes
```

PacienteClinicoDTO.java

```
Campos: id, nome, cpf, dataNascimento, telefone, prontuarios[], receitas[], atestados[]
Uso: Visão clínica completa do paciente (prontuários, receitas, atestados)
```

MedicoDTO.java

```
Campos: id, crm, nomeUsuario, valorConsulta, especialidades[]
Uso: Listagem e detalhes de médicos
```

ProntuarioDTO.java

```
Campos: id, consultaId, diagnosticoCid, evolucaoClinica, condutaMedica, observacoes
Uso: Criação e visualização de prontuários
```

PagamentoDTO.java

```
Campos: id, consultaId, valor, status, formaPagamento
Uso: Visualização e operações de pagamento
```

TriagemDTO.java

```
Campos: id, consultaId, pressaoArterial, temperatura, frequenciaCardiaca, peso, altura, saturacao,
queixaPrincipal, prioridade
Uso: Registro de triagem pela equipe de enfermagem
```

ReceitaDTO.java

```
Campos: id, pacienteId, medicamento, dosagem, instrucoes, dataEmissao
Uso: Prescrição de medicamentos
```

AtestadoDTO.java

```
Campos: id, pacienteId, descricao, dias, dataEmissao
Uso: Emissão de atestados
```

AuditoriaDTO.java

```
Campos: id, usuarioId, usuarioEmail, acao, entidade, entidadeId, detalhes, dataHora
Uso: Visualização de logs de auditoria
```

FaturamentoRelatorioDTO.java

```
Campos: totalFaturadoDia, totalFaturadoMes, pagamentosPendentesCount, pagamentosRecebidosCount,
totalPendente
Uso: Relatório financeiro para o setor de faturamento
```

FuncionarioRequest.java

```
Campos: nome, email, senha, cargo, matricula, dataAdmissao
Uso: Criação de funcionários pelo admin
```

4.3. Camada de Repositórios (repository/)

O que são Repositórios? São interfaces que fazem a ponte entre o código Java e o banco de dados. O Spring Data JPA implementa automaticamente métodos baseados no nome do método.

Analogia: É como ter um bibliotecário que entende pedidos em português. Você fala "encontre por email" (`findByEmail`) e ele sabe exatamente como buscar no banco.

Métodos importantes (além dos padrões findById, findAll, save, delete):

Repositório	Método Personalizado	O que faz
UsuarioRepository	`findByEmail(String email)`	Busca usuário pelo e-mail
ConsultaRepository	`existsByMedicoIdAndDataHoraBetween(UUID, LocalDateTime, LocalDateTime)`	Verifica conflito de horário do médico
ConsultaRepository	`existsByPacienteIdAndDataHoraBetween(UUID, LocalDateTime, LocalDateTime)`	Verifica conflito de horário do paciente
ConsultaRepository	`findByDataHoraBetween(LocalDateTime, LocalDateTime)`	Lista consultas de um período
ConsultaRepository	`findByMedicoIdAndDataHoraBetween(UUID, LocalDateTime, LocalDateTime)`	Agenda do dia de um médico
PagamentoRepository	`findByConsultaId(UUID)`	Busca pagamento de uma consulta
ProntuarioRepository	`findByConsultaId(UUID)`	Busca prontuário de uma consulta
ProntuarioRepository	`findByConsultaPacienteId(UUID)`	Todos os prontuários de um paciente
TriagemRepository	`findByConsultaId(UUID)`	Busca triagem de uma consulta
MedicoRepository	`findByUsuarioEmail(String)`	Busca médico pelo e-mail do usuário
ReceitaRepository	`findByPacienteId(UUID)`	Todas receitas de um paciente
AtestadoRepository	`findByPacienteId(UUID)`	Todos atestados de um paciente

4.4. Camada de Serviços (service/) — LÓGICA DE NEGÓCIO

Esta é a camada mais importante. Aqui vivem TODAS as regras do sistema.

AuthService.java — Autenticação

Método `register(RegisterRequest request)` :

1. Verifica se o e-mail já existe no banco → se sim, lança erro
2. Cria um novo `Usuario` com:
   - Nome, email do request
   - Senha criptografada com BCrypt (nunca salva em texto puro!)

    - Role = "PACIENTE" (todo novo cadastro é paciente)
3. Salva o usuário no banco
4. Cria um novo `Paciente` vinculado ao usuário
5. Salva o paciente
6. Gera um token JWT
7. Retorna o token e a role

Método `login(LoginRequest request)`:

1. Usa o AuthenticationManager do Spring para validar email + senha
2. Busca o usuário pelo email
3. Gera um token JWT
4. Retorna o token e a role

ConsultaService.java — Agendamento de Consultas

Método `agendar(ConsultaDTO dto)` — O mais complexo:

1. Valida que a data é no futuro (mínimo 30min de antecedência)
2. Busca o médico pelo ID → se não existe, erro
3. Busca o paciente pelo ID → se não existe, erro
4. **Verificação de ownership:** Se quem está logado é PACIENTE, só pode agendar para si mesmo
5. **Verificação de conflito de horário do médico:** Verifica se o médico já tem consulta no intervalo de ±59 minutos
6. **Verificação de conflito de horário do paciente:** Verifica se o paciente já tem consulta no mesmo intervalo
7. Cria a consulta com status "AGENDADA"
8. Cria automaticamente um Pagamento "PENDENTE" com o valor da consulta do médico
9. Envia notificação via RabbitMQ
10. Registra na auditoria
11. Retorna o DTO da consulta

Método `remarcar(UUID id, LocalDateTime novaDataHora)`:

1. Valida antecedência mínima de 30 minutos
2. Verifica que a consulta está AGENDADA
3. Verifica ownership (paciente só remarca a sua)
4. Valida conflitos de horário
5. Atualiza a data
6. Notifica e audita

Método `checkin(UUID id)`:

1. Busca a consulta
2. Verifica status válido (não pode ser CANCELADA ou REALIZADA)
3. Muda status para "AGUARDANDO_TRIAGEM"
4. Notifica e audita

Método `cancelar(UUID id)`:

1. Verifica ownership
2. Valida antecedência de 2 horas
3. Muda status para "CANCELADA"

4. Notifica e audita

Método `realizar(UUID id)`:

1. Verifica que não está cancelada
2. Verifica que o pagamento está "PAGO"
3. Muda status para "REALIZADA"
4. Audita

Método `agendaHoje()`:

1. Identifica o médico logado pelo email do JWT
2. Busca consultas do dia
3. Ordena por prioridade de triagem: EMERGENCIA > ALTA > MEDIA > BAIXA

PagamentoService.java — Gestão de Pagamentos

Método `pagar(UUID id)`:

1. Busca o pagamento pelo ID
2. Valida que está PENDENTE
3. Muda status para "PAGO"
4. Registra na auditoria

Método `estornar(UUID id)`:

1. Busca o pagamento
2. Valida que está PAGO
3. Muda status para "ESTORNADO"
4. Audita

Método `findById(UUID id)`:

1. Busca o pagamento
2. Se quem pede é PACIENTE, verifica que é seu (ownership check)

FaturamentoService.java — Relatórios Financeiros

Método `gerarRelatorio()`:

1. Busca TODOS os pagamentos do banco
2. Calcula:
   - Total faturado no dia (pagamentos PAGO com data de hoje)
   - Total faturado no mês (pagamentos PAGO do mês atual)
   - Quantidade de pendentes e recebidos
   - Total em reais pendente

AuditoriaService.java — Registro de Ações

Método `registrarAcao(String acao, String entidade, UUID entidadeId, String detalhes)`:

1. Pega o usuário logado do SecurityContext
2. Busca o `Usuario` completo pelo email
3. Cria um registro de `Auditoria` com todos os dados
4. Salva no banco
5. Se der erro, captura silenciosamente (nunca trava o fluxo principal)

TriagemService.java — Triagem de Enfermagem

Método `registrarTriagem(TriagemDTO dto)`:

1. Busca a consulta associada
2. Valida que o status é "AGUARDANDO_TRIAGEM"
3. Cria o registro com sinais vitais e prioridade
4. Muda status da consulta para "TRIADA"

PacienteService.java — Gestão de Pacientes

Inclui lógica para:

- Criação inline de paciente + usuário (pela recepção)
- Prescrição de receitas (`prescreverReceita`)
- Emissão de atestados (`emitirAtestado`)
- Visualização clínica completa (`mapToClinicoDTO`) com prontuários, receitas e atestados

NotificacaoListener.java — Consumidor de Mensagens RabbitMQ

Escuta a fila `notificacoes` e imprime no console. Aqui é onde, no futuro, se integraria com serviço de e-mail, SMS ou push notification.

4.5. Camada de Controllers (controller/) — ROTAS HTTP

Mapa COMPLETO de todas as rotas da API:

Autenticação (AuthController — /auth)

Método	Rota	Descrição	Quem pode
POST	`/auth/login`	Fazer login	Todos (público)
POST	`/auth/register`	Cadastrar paciente	Todos (público)

Admin (AdminController — /admin)

Método	Rota	Descrição	Quem pode
POST	`/admin/funcionarios`	Criar funcionário	ADMIN

Consultas (ConsultaController — /api/consultas)

Método	Rota	Descrição	Quem pode
POST	`/api/consultas`	Agendar consulta	PACIENTE, RECEPCIONISTA, ADMIN
GET	`/api/consultas`	Listar todas	ADMIN, RECEPCIONISTA
PUT	`/api/consultas/{id}/remarcar`	Remarcar	PACIENTE, RECEPCIONISTA, ADMIN
PUT	`/api/consultas/{id}/confirmar`	Confirmar	PACIENTE
PUT	`/api/consultas/{id}/checkin`	Check-in na recepção	RECEPCIONISTA, ADMIN

Método	Rota	Descrição	Quem pode
PUT	`/api/consultas/{id}/cancelar`	Cancelar	PACIENTE, RECEPCIONISTA, ADMIN
PUT	`/api/consultas/{id}/realizar`	Marcar como realizada	MEDICO
GET	`/api/consultas/hoje`	Agenda do dia	RECEPCIONISTA, ADMIN
GET	`/api/consultas/medico/agenda-hoje`	Agenda do médico logado	MEDICO

Pacientes (PacienteController — /api/pacientes)

Método	Rota	Descrição	Quem pode
POST	`/api/pacientes`	Criar paciente (recepção)	RECEPCIONISTA, ADMIN
GET	`/api/pacientes`	Listar todos	ADMIN, RECEPCIONISTA
GET	`/api/pacientes/{id}`	Detalhes de um paciente	Autenticado
GET	`/api/pacientes/{id}/clinico`	Visão clínica completa	MEDICO, PACIENTE
POST	`/api/pacientes/{id}/receitas`	Prescrever receita	MEDICO
POST	`/api/pacientes/{id}/atestados`	Emitir atestado	MEDICO

Médicos (MedicoController — /api/medicos)

Método	Rota	Descrição	Quem pode
POST	`/api/medicos`	Cadastrar médico	ADMIN
GET	`/api/medicos`	Listar todos	Autenticado
GET	`/api/medicos/{id}`	Detalhes	Autenticado

Prontuários (ProntuarioController — /api/prontuarios)

Método	Rota	Descrição	Quem pode
POST	`/api/prontuarios`	Criar/editar prontuário	MEDICO
GET	`/api/prontuarios/historico`	Histórico (médico vê todos, paciente só os seus)	MEDICO, PACIENTE
GET	`/api/prontuarios/{id}`	Detalhes de um prontuário	MEDICO, PACIENTE

Pagamentos (PagamentoController — /api/pagamentos)

Método	Rota	Descrição	Quem pode
POST	`/api/pagamentos/{id}/pagar`	Registrar pagamento	FATURAMENTO, ADMIN, RECEPCIONISTA
POST	`/api/pagamentos/{id}/estornar`	Estornar pagamento	FATURAMENTO, ADMIN
GET	`/api/pagamentos/{id}`	Ver pagamento	FATURAMENTO, ADMIN, RECEPCIONISTA, PACIENTE

Triagem (via ConsultaController)

Método	Rota	Descrição	Quem pode
POST	`/api/triagem`	Registrar triagem	TRIAGEM

Especialidades (EspecialidadeController — /api/especialidades)

Método	Rota	Descrição	Quem pode
POST	`/api/especialidades`	Criar especialidade	ADMIN
GET	`/api/especialidades`	Listar todas	Autenticado

Exames (ExameController — /api/exames)

Método	Rota	Descrição	Quem pode
POST	`/api/exames`	Solicitar exame	MEDICO
GET	`/api/exames`	Listar exames	MEDICO, ADMIN
GET	`/api/exames/{id}`	Detalhes	Autenticado

Faturamento (FaturamentoController — /api/faturamento)

Método	Rota	Descrição	Quem pode
GET	`/api/faturamento/relatorio`	Relatório financeiro	FATURAMENTO, ADMIN

Auditoria (AuditoriaController — /api/auditoria)

Método	Rota	Descrição	Quem pode
GET	`/api/auditoria?limite=100`	Últimos logs do sistema	ADMIN

Endpoints de Infraestrutura (públicos)

Método	Rota	Descrição
GET	`/actuator/health`	Status de saúde da aplicação
GET	`/actuator/prometheus`	Métricas para Prometheus
GET	`/swagger-ui.html`	Documentação interativa da API
GET	`/v3/api-docs`	Especificação OpenAPI em JSON

PARTE 5 — SEGURANÇA: COMO FUNCIONA A PROTEÇÃO

5.1. Fluxo de Autenticação (Login)

```
1. Usuário envia POST /auth/login com { email, senha }
2. AuthService valida as credenciais com BCrypt
3. JwtService gera um token JWT contendo:
   - Subject: email do usuário
   - Role: papel do usuário
   - Emitido em: timestamp atual
   - Expira em: 24 horas depois
4. Sistema retorna { token: "eyJhbG...", role: "PACIENTE" }
5. Frontend salva o token no localStorage do navegador
```

5.2. Fluxo de Autorização (Acessar recurso protegido)

```
1. Frontend faz requisição com header: Authorization: Bearer eyJhbG...
2. JwtAuthenticationFilter intercepta a requisição
3. Extrai o token do header
4. JwtService valida:
   - Assinatura (não foi adulterado?)
   - Expiração (ainda é válido?)
5. Extrai o email do token
6. Busca o usuario completo no banco
7. Cria um contexto de segurança com as authorities (roles)
8. A requisição segue para o Controller
9. @PreAuthorize verifica se a role do usuário permite a ação
10. Se sim: executa. Se não: retorna 403 Forbidden.
```

5.3. Tabela de Permissões por Role

Funcionalidade	ADMIN	MEDICO	PACIENTE	RECEPCIONISTA	TRIAGEM	FATURAMENTO
Criar funcionário	✅	❌	❌	❌	❌	❌
Criar médico	✅	❌	❌	❌	❌	❌
Agendar consulta	✅	❌	✅ (só p/ si)	✅	❌	❌
Remarcar consulta	✅	❌	✅ (só a sua)	✅	❌	❌
Check-in	✅	❌	❌	✅	❌	❌
Cancelar consulta	✅	❌	✅ (só a sua)	✅	❌	❌
Realizar consulta	❌	✅	❌	❌	❌	❌
Fazer triagem	❌	❌	❌	❌	✅	❌
Criar prontuário	❌	✅	❌	❌	❌	❌
Ver prontuário	❌	✅ (todos)	✅ (só os seus)	❌	❌	❌
Prescrever receita	❌	✅	❌	❌	❌	❌

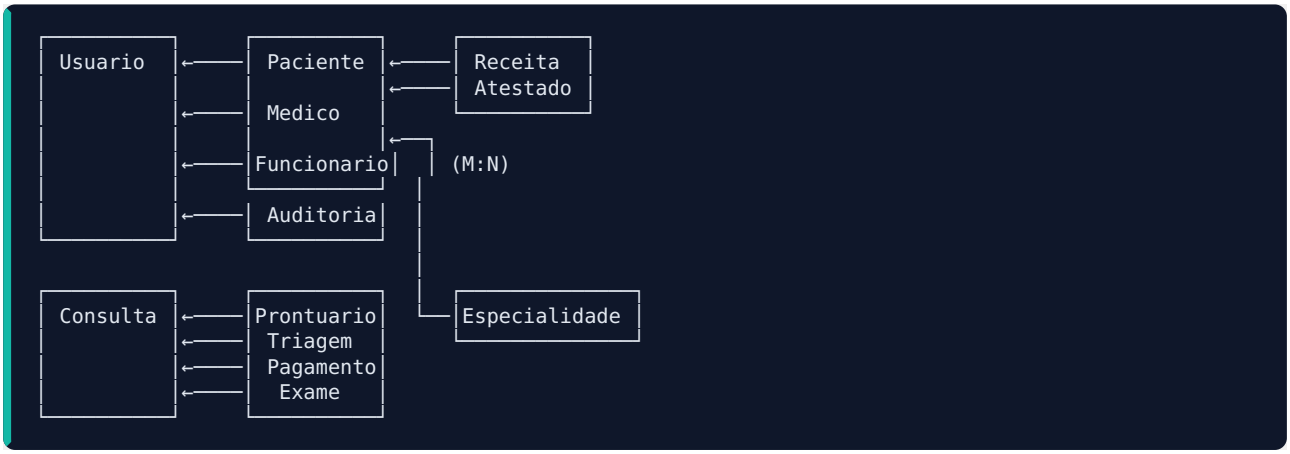
Funcionalidade	ADMIN	MEDICO	PACIENTE	RECEPCIONISTA	TRIAGEM	FATURAMENTO
Emitir atestado	❌	✅	❌	❌	❌	❌
Pagar consulta	✅	❌	❌	✅	❌	✅
Estornar pagamento	✅	❌	❌	❌	❌	✅
Relatório faturamento	✅	❌	❌	❌	❌	✅
Ver auditoria	✅	❌	❌	❌	❌	❌
Swagger/API docs	✅	✅	✅	✅	✅	✅

5.4. Senhas e Criptografia

- Senhas NUNCA são salvas em texto puro. São criptografadas com **BCrypt** antes de ir para o banco.
- BCrypt é um algoritmo de hash de mão única: é impossível reverter o hash para descobrir a senha original.
- O token JWT é assinado com uma chave secreta definida no `application.yml`.

PARTE 6 — BANCO DE DADOS: ESTRUTURA DAS TABELAS

6.1. Diagrama de Relacionamento



6.2. Migrações SQL (na ordem de execução)

V1 — Tabelas Iniciais

Cria: `usuario`, `paciente`, `medico`, `consulta`, `prontuario`, `pagamento`, `exame`, `especialidade`

V2 — Funcionários

Cria: `funcionario` (para recepcionistas, triagem, faturamento)

V3 — Especialidade Médica (M:N)

Cria: `medico_especialidade` (tabela de junção muitos-para-muitos)

V4 — Limpeza

Remove coluna `especialidade` da tabela `medico` (migrada para M:N)

V5 — Triagem

Cria: `triagem` (sinais vitais, prioridade)

V6 — Receitas e Atestados

Cria: `receita`, `atestado`. Adiciona colunas `diagnostico_cid`, `evolucao_clinica`, `conduta_medica` ao `prontuario`

V7 — Auditoria

Cria: `auditoria` (log de ações do sistema)

PARTE 7 — FRONTEND: COMO FUNCIONA A INTERFACE

7.1. Tecnologias do Frontend

Tecnologia	Versão	Função
React	18	Biblioteca de UI (construir telas)
TypeScript	-	Linguagem com tipos (menos erros)
Vite	-	Build tool (compilação rápida)
Tailwind CSS	3	Framework de CSS (estilização rápida)
Axios	-	Cliente HTTP (comunicação com backend)
React Router DOM	6	Navegação entre páginas sem recarregar

7.2. Páginas

Login.tsx

- Formulário com email e senha
- Chama POST `/auth/login`
- Salva o token no contexto e localStorage
- Redireciona para o Dashboard

Register.tsx

- Formulário com nome, email, senha, CPF, data de nascimento, telefone
- Chama POST `/auth/register`
- Cria o paciente e faz login automático

Dashboard.tsx

- Painel principal após login
- Mostra informações baseadas na role do usuário

Medicos.tsx

- Lista todos os médicos
- Mostra CRM, especialidades e valor da consulta

MinhasConsultas.tsx

- Lista as consultas do paciente logado
- Permite ver status e detalhes

7.3. Componentes de Infraestrutura

AuthContext.tsx

- Gerencia o estado de autenticação globalmente
- Armazena token e role
- Oferece funções `login()` e `logout()`
- Persiste no localStorage (não perde ao recarregar a página)

ProtectedRoute.tsx

- Componente que envolve páginas protegidas
- Se não está logado, redireciona para `/login`

Layout.tsx

- Layout com menu lateral (sidebar)
- Adapta os itens de menu baseado na role do usuário

axios.ts

- Configura o Axios com baseURL apontando para o backend
- Intercepta todas as requisições para adicionar o header `Authorization: Bearer <token>` automaticamente

7.4. Build e Deploy

No Dockerfile do frontend:

1. **Estágio de build:** Usa Node.js para instalar dependências e compilar (`npm run build`)
2. **Estágio de produção:** Copia os arquivos compilados para o Nginx
3. O `nginx.conf` configura:
    - Serve arquivos estáticos na porta 80
    - Redireciona `/api/*` para o backend (proxy reverso)
    - Suporta SPA (Single Page Application) com fallback para `index.html`

PARTE 8 — INFRAESTRUTURA: DOCKER, KUBERNETES E MONITORAMENTO

8.1. Docker Compose (docker-compose.yml)

O arquivo `docker-compose.yml` define 6 serviços:

Serviço `db` (PostgreSQL)

- Porta: 5432
- Volume persistente: `medflow-postgres-data` (dados não se perdem ao reiniciar)
- Healthcheck: verifica se o PostgreSQL está pronto antes de liberar outros serviços

Serviço `rabbitmq` (RabbitMQ)

- Porta AMQP: 5672, Porta Web: 15672
- Usa tmpfs (armazenamento em memória) para melhor performance
- Healthcheck com timeout de 30s no start (demora a subir)

Serviço `backend` (Spring Boot)

- Porta: 8080
- Depende de `db` e `rabbitmq` estarem saudáveis
- Variáveis de ambiente configuram conexão com DB e RabbitMQ
- Profile ativo: `dev`

Serviço `frontend` (Nginx + React)

- Porta: 5173 → 80 (mapeada para 5173 no host por conveniência)
- Depende do backend estar rodando

Serviço `prometheus` (Monitoramento)

- Porta: 9090
- Monta o arquivo `prometheus.yml` como configuração
- Coleta métricas do backend a cada 15 segundos

Serviço `grafana` (Dashboards)

- Porta: 3000
- Depende do Prometheus
- Credenciais: admin/admin

Como subir tudo:

```
docker compose up -d
```

Este único comando sobe os 6 containers na ordem correta.

8.2. Kubernetes (k8s/)

Os arquivos Kubernetes são para deploy em produção (nuvem). Cada arquivo define:

backend.yaml

- Deployment: 2 réplicas do backend
- Service: Expõe na porta 8080
- ConfigMap: Variáveis de ambiente

frontend.yaml

- Deployment: 2 réplicas do frontend
- Service: Expõe na porta 80
- Pode ser configurado como LoadBalancer para acesso externo

postgres.yaml

- StatefulSet: 1 réplica (banco de dados não escala horizontalmente)
- PersistentVolumeClaim: 5GB de armazenamento persistente
- Service: Expõe na porta 5432 internamente

rabbitmq.yaml

- Deployment: 1 réplica
- Service: Portas 5672 e 15672

8.3. Prometheus (prometheus/prometheus.yml)

Configuração que define:

- Intervalo de scraping: 15 segundos
- Target: `backend:8080` (endpoint `/actuator/prometheus`)
- Métricas coletadas: HTTP requests, JVM memory, database connections, etc.

PARTE 9 — FLUXOS COMPLETOS DO SISTEMA

9.1. Fluxo Completo: Paciente faz consulta do início ao fim

```
1. PACIENTE acessa o site e clica em "Cadastrar"
   → POST /auth/register
   → Sistema cria Usuario (role=PACIENTE) + Paciente
   → Retorna token JWT

2. PACIENTE faz login
   → POST /auth/login
   → Retorna token JWT

3. PACIENTE agenda consulta
   → POST /api/consultas
   → Sistema valida conflitos de horário
   → Cria Consulta (status=AGENDADA)
   → Cria Pagamento (status=PENDENTE)
   → Envia notificação via RabbitMQ
   → Registra na auditoria

4. PACIENTE confirma a consulta
   → PUT /api/consultas/{id}/confirmar
   → Status muda para CONFIRMADA

5. PACIENTE chega na clínica, RECEPCIONISTA faz check-in
   → PUT /api/consultas/{id}/checkin
   → Status muda para AGUARDANDO_TRIAGEM

6. ENFERMEIRO DE TRIAGEM registra sinais vitais
   → POST /api/triagem
   → Registra pressão, temperatura, peso, etc.
   → Define prioridade: BAIXA, MEDIA, ALTA, EMERGENCIA
   → Status da consulta muda para TRIADA

7. FATURAMENTO registra pagamento
   → POST /api/pagamentos/{id}/pagar
   → Status do pagamento muda para PAGO
   → Registra na auditoria

8. MÉDICO atende o paciente (na agenda do dia, ordenada por prioridade)
   → GET /api/consultas/medico/agenda-hoje
   → Pacientes com EMERGENCIA aparecem primeiro

9. MÉDICO registra o prontuário
   → POST /api/prontuarios
   → Registra diagnóstico CID, evolução clínica, conduta médica

10. MÉDICO prescreve medicamento
    → POST /api/pacientes/{id}/receitas
    → Registra medicamento, dosagem, instruções

11. MÉDICO emite atestado (se necessário)
    → POST /api/pacientes/{id}/atestados
    → Registra descrição e dias de afastamento

12. MÉDICO marca consulta como realizada
    → PUT /api/consultas/{id}/realizar
    → Status muda para REALIZADA (só funciona se pagamento estiver PAGO)

13. ADMIN consulta a auditoria
    → GET /api/auditoria
    → Vê todo o rastro de quem fez o quê, quando

14. FATURAMENTO consulta relatório
```

```
    → GET /api/faturamento/relatorio
    → Vê total faturado no dia, mês, pendentes
```

PARTE 10 — CREDENCIAIS E DADOS DE ACESSO

10.1. Banco de Dados PostgreSQL

- **Host:** localhost (ou `db` dentro do Docker)
- **Porta:** 5432
- **Banco:** medflow
- **Usuário:** medflow
- **Senha:** medflow

Comando para acessar diretamente:

```
docker exec -it medflow-db psql -U medflow -d medflow
```

10.2. RabbitMQ

- **Painel Web:** http://localhost:15672
- **Usuário:** guest
- **Senha:** guest

10.3. Grafana

- **URL:** http://localhost:3000
- **Usuário:** admin
- **Senha:** admin

10.4. Prometheus

- **URL:** http://localhost:9090

10.5. Swagger (Documentação da API)

- **URL:** http://localhost:8080/swagger-ui.html

10.6. Frontend (Interface Web)

- **URL:** http://localhost:5173

10.7. Backend (API REST)

- **URL:** http://localhost:8080

10.8. Usuário Admin Padrão

- **Email:** admin@medflow.com
- **Senha:** admin

10.9. Chave Secreta JWT

Definida no arquivo `application.yml`:

```
jwt.secret: minha-chave-secreta-super-segura-para-jwt-do-medflow
```

Em produção, DEVE ser trocada por uma chave aleatória forte.

PARTE 11 — COMO RODAR O SISTEMA

11.1. Pré-requisitos

1. **Docker Desktop** instalado e rodando
2. **Git** instalado

11.2. Passo a passo

```
# 1. Clonar o repositório
git clone https://github.com/Samuka310/medflow.git

# 2. Entrar na pasta
cd medflow

# 3. Subir todos os containers
docker compose up -d

# 4. Aguardar ~60 segundos para tudo inicializar

# 5. Acessar
# Frontend: http://localhost:5173
# Backend/API: http://localhost:8080
# Swagger: http://localhost:8080/swagger-ui.html
# RabbitMQ: http://localhost:15672
# Grafana: http://localhost:3000
# Prometheus: http://localhost:9090
```

11.3. Parar o sistema

```
docker compose down
```

11.4. Parar e APAGAR todos os dados

```
docker compose down -v
```

O `-v` remove os volumes (dados do banco serão perdidos).

PARTE 12 — GLOSSÁRIO DE TERMOS TÉCNICOS

Termo	Significado
API	Interface de Programação de Aplicações — conjunto de rotas que o frontend chama para se comunicar com o backend
REST	Estilo de arquitetura para APIs que usa verbos HTTP (GET, POST, PUT, DELETE)
JSON	Formato de dados usado na comunicação entre frontend e backend. Ex: `{"nome": "João"}`
JWT	Token de autenticação — uma string criptografada que prova quem é o usuário
UUID	Identificador Único Universal — um ID alfanumérico de 36 caracteres que nunca se repete
CRUD	Create, Read, Update, Delete — as 4 operações básicas de dados
ORM	Object-Relational Mapping — tecnologia que converte objetos Java em tabelas SQL
MVC	Model-View-Controller — padrão de organização de código
Container	Ambiente isolado e leve que contém tudo para rodar uma aplicação
Migration	Script SQL versionado que altera a estrutura do banco de dados
Endpoint	Um ponto de entrada da API. Ex: `/api/consultas` é um endpoint
Middleware	Código que intercepta requisições antes de chegarem ao destino final
BCrypt	Algoritmo de criptografia para senhas
CORS	Cross-Origin Resource Sharing — regra de segurança do navegador
Proxy Reverso	Servidor intermediário que redireciona requisições para outros servidores
Healthcheck	Verificação automática de saúde de um serviço
Scraping	Coleta automática de dados (neste caso, métricas do Prometheus)
SPA	Single Page Application — site que não recarrega ao navegar
Role	Papel/função de um usuário no sistema (ADMIN, MEDICO, etc.)
Bearer Token	Formato de envio do JWT: `Authorization: Bearer <token>`

Fim do Documento

Este documento cobre 100% da estrutura atual do MedFlow versão 2.9. Qualquer adição futura (Fases 2.10, 2.11, 2.12) será adicionada como apêndice.